

React Custom Hooks Lab: Extract, Build & Compose

MERN Stack Bootcamp – Hands-On Practice

Instructor: Kashif Raza | [LinkedIn](#)

Duration: ~2 hours

Difficulty: Intermediate

Lab Overview

You will extract inline logic into **4 production-grade custom hooks**, compose them, and verify behavior. Each phase builds independently. Focus on **cleanup**, **dependency arrays**, and **SSR safety**.

Project Setup

```
npm create vite@latest hooks-lab -- --template react
cd hooks-lab
npm install
npm install eslint-plugin-react-hooks --save-dev
npm run dev
```

`.eslintrc.js`

```
module.exports = {
  plugins: ['react-hooks'],
  rules: {
    'react-hooks/rules-of-hooks': 'error',
    'react-hooks/exhaustive-deps': 'warn'
  }
};
```

◆ Phase 1: Extract useLocalStorage

Goal: Replace repetitive `useState` + `useEffect` persistence logic with a reusable hook.

Starter Code (components/ThemeToggle.jsx)

```
import { useState, useEffect } from 'react';

export default function ThemeToggle() {
  const [theme, setTheme] = useState(() => {
    return localStorage.getItem('theme') || 'light';
  });

  useEffect(() => {
    localStorage.setItem('theme', theme);
    document.documentElement.setAttribute('data-theme', theme);
  }, [theme]);

  return (
    <button onClick={() => setTheme(t => t === 'light' ? 'dark' : 'light')}>
      Switch to {theme === 'light' ? 'Dark' : 'Light'} Mode
    </button>
  );
}
```

Instructions

1. Create `hooks/useLocalStorage.js`
2. Implement safe SSR checks (`typeof window !== 'undefined'`)
3. Handle JSON parsing errors and functional updates
4. Refactor `ThemeToggle` to use `const [theme, setTheme] = useLocalStorage('theme', 'light')`

✅ Success Criteria:

- ☐ Works on first load without hydration mismatch
- ☐ Persists across refresh
- ☐ Handles `localStorage` quota/full storage gracefully
- ☐ ESLint passes with no hook warnings

◆ Phase 2: Build `useDebounce` + `useFetch`

Goal: Implement two independent hooks, then verify they handle async timing and cleanup correctly.

Starter Code (`components/ApiSearch.jsx`)

```
import { useState, useEffect } from 'react';

export default function ApiSearch() {
  const [query, setQuery] = useState('');
  const [results, setResults] = useState([]);
  const [loading, setLoading] = useState(false);

  // ✖ BAD: Fires on every keystroke, no abort, race conditions
  useEffect(() => {
    if (!query) return;
    setLoading(true);
    fetch(`https://jsonplaceholder.typicode.com/posts?title_like=${query}`)
      .then(res => res.json())
      .then(data => { setResults(data); setLoading(false); });
  }, [query]);

  return (
    <div>
      <input value={query} onChange={e => setQuery(e.target.value)} placeholder="Search posts..." />
      {loading && <p>Loading...</p>}
      <ul>{results.map(r => <li key={r.id}>{r.title}</li>)}</ul>
    </div>
  );
}
```

Instructions

1. Create `hooks/useDebounce.js` → delay state updates until typing stops
2. Create `hooks/useFetch.js` → handle loading, error, `AbortController`, and retry
3. Refactor `ApiSearch` to use both hooks
4. Add a "Retry" button that triggers refetch on error

✓ Success Criteria:

- ☐ Only 1 API call after typing pauses (check Network tab)
- ☐ Fast typing doesn't cause race conditions (`AbortController` works)
- ☐ Unmounting during fetch doesn't trigger "setState on unmounted component" warning
- ☐ Error state displays cleanly

◆ Phase 3: useClickOutside + Hook Composition

Goal: Build an interactive dropdown that closes on outside click, then compose hooks into a single `useSearch` hook.

Starter Code (`components/DropdownMenu.jsx`)

```
import { useState, useRef, useEffect } from 'react';

export default function DropdownMenu() {
  const [open, setOpen] = useState(false);
  const dropdownRef = useRef(null);

  // ✖ BAD: Manual event listener, no cleanup, captures stale state
  useEffect(() => {
    document.addEventListener('mousedown', (e) => {
      if (!dropdownRef.current.contains(e.target)) setOpen(false);
    });
  }, []);

  return (
    <div ref={dropdownRef} className="relative">
      <button onClick={() => setOpen(o => !o)}>Options ▼</button>
      {open && (
        <div className="absolute top-full left-0 bg-white border shadow p-2">
          <button className="block w-full text-left p-1">Edit</button>
          <button className="block w-full text-left p-1">Delete</button>
        </div>
      )}
    </div>
  );
}
```

Instructions

1. Create `hooks/useClickOutside.js` → return `ref`, handle `mousedown` with proper cleanup
2. Use `useRef` to store the latest handler (prevents stale closure without re-attaching listeners)
3. Compose into `hooks/useSearch.js` that exports `{ query, setQuery, results, loading, error, clear }`
4. Replace `ApiSearch` and `DropdownMenu` logic with composed hooks

✓ Success Criteria:

- ☐ Dropdown closes when clicking outside, but NOT inside
- ☐ Event listener is properly removed on unmount
- ☐ `useSearch` returns a clean object, hides implementation details
- ☐ React DevTools shows hooks in correct order



Phase 4: Testing Custom Hooks (Lightweight)

Goal: Learn to test hooks without heavy mocking.



Setup

```
npm install @testing-library/react @testing-library/jest-dom vitest jsdom --save-dev
```

vite.config.js

```
import { defineConfig } from 'vite'
import react from '@vitejs/plugin-react'
export default defineConfig({
  plugins: [react()],
  test: { environment: 'jsdom' }
})
```



Test File (__tests__/useDebounce.test.jsx)

```
import { renderHook, waitFor } from '@testing-library/react';
import { useDebounce } from '../hooks/useDebounce';

describe('useDebounce', () => {
  vi.useFakeTimers(); // Vitest/Jest fake timers

  it('debounces value updates', () => {
    const { result, rerender } = renderHook(
      ({ value }) => useDebounce(value, 300),
      { initialProps: { value: 'fast' } }
    );

    expect(result.current).toBe('fast');
    rerender({ value: 'too-fast' });
    expect(result.current).toBe('fast'); // Still old value

    vi.advanceTimersByTime(300);
    waitFor(() => expect(result.current).toBe('too-fast'));
  });
});
```



Instructions

1. Run `npm test` → verify it passes
2. Write 1 test for `useLocalStorage` (mock `window.localStorage` if needed)
3. Verify hook cleanup by checking listener/timer removal



Success Criteria:

- ☐ Test suite runs green
- ☐ Debounce delay is respected in test
- ☐ No async/unhandled promise warnings

Troubleshooting Guide

Symptom	Cause	Fix
Hook not updating UI	Returning stale reference or mutating state	Always return new objects/arrays
Memory leak warning	Missing cleanup in <code>useEffect</code>	Return cleanup function; clear timers/abort fetches
Stale closure in event listener	Listener captures old state	Store handler in <code>useRef</code> or include in deps
SSR hydration mismatch	Accessing <code>window</code> during render	Wrap browser APIs in <code>typeof window !== 'undefined'</code> or lazy init
ESLint <code>exhaustive-deps</code> warning	Missing dependency	Add variable to array OR memoize it with <code>useCallback</code> / <code>useMemo</code>

Debug Pro Tips

```
// 1. Track hook execution order
useEffect(() => console.log('Hook mounted/cleaned'));

// 2. Verify memoization stability
const fn = useCallback(() => {}, []);
console.log(fn === prevFn); // Should be true across renders

// 3. React DevTools → Components → Expand "Hooks"
// Inspect live state, verify ref values, check dependency arrays
```



Submission Guidelines



Deliverables

1. **GitHub Repository** containing:

- `src/hooks/` with all 4 implemented hooks
- `src/components/` using the hooks (no inline `useEffect` for these patterns)
- `__tests__/` with at least 2 hook tests
- `README.md` with:
 - Hook signatures & return types
 - Screenshot/GIF of working demo
 - "Lessons Learned" section (1 paragraph)



Evaluation Criteria

Criteria	Points
✓ Hooks follow Rules of Hooks (ESLint clean)	30
✓ Proper cleanup & AbortController usage	25
✓ Composition pattern implemented cleanly	20
✓ Tests pass & cover edge cases	15
✓ Code documented with JSDoc	10
Total	100



Bonus (+10)

- Add TypeScript definitions (`index.d.ts` or JSDoc `@template`)
- Implement `useIntersectionObserver` hook for infinite scroll
- Deploy to Vercel and share live link in README

💬 Instructor Note:

Custom hooks are your superpower. They turn scattered `useEffect` spaghetti into predictable, testable modules. When you finish this lab, audit one component from your portfolio. Extract its logic. Name it. Test it. Import it. That's how senior engineers ship.

Push your repo, tag me in your PR, and I'll leave architecture feedback. Keep your dependencies explicit, your cleanup tight, and your hooks focused on **one responsibility**.

— Kashif Raza | Composable logic, production-ready UI 🧡

Lab Version: 1.0 | Last Updated: \$(date)

© 2025 MERN Stack Bootcamp. For educational use only.